

# Desarrollo en el cliente

## Juegos en Red - Grado en Diseño y Desarrollo de Videojuegos

Ruben Rodríguez

ruben.rodriguez@urjc.es

URJC

Natalia Madrueño

natalia.madrueño@urjc.es

URJC

Mario Manzano

mario.manzano@urjc.es

URJC

2026-09-09



# Tabla de contenidos

- [Introducción a JavaScript](#)
- [Configuración del Entorno](#)
- [El Lenguaje JavaScript](#)
- [Arrays](#)
- [Control de Flujo](#)
- [Funciones](#)
- [Manejo de Excepciones](#)
- [Almacenamiento de Datos](#)



# Introducción a JavaScript

# Introducción

JavaScript es el lenguaje fundamental para desarrollo web interactivo y videojuegos web.

## Características principales:

- Scripting (no necesita compilador)
- Tipado dinámico
- Funcional
- Orientado a objetos (prototipos)

## Aplicaciones:

- Interactividad en páginas web
- Modificación del DOM
- Peticiones AJAX
- Videojuegos web



# Versiones de ECMAScript

- Primera versión en 10 días (1995).
- **ES5 (2009)**: Base sólida que estableció JavaScript moderno en todos los navegadores
- **ES6/ES2015**: Revolución del lenguaje - introdujo sintaxis moderna que cambió cómo programamos
- **Actualizaciones anuales**: Cada año se añaden mejoras sin romper código existente (compatibilidad hacia atrás)
- **ES2015 marcó un antes y después**: La mayoría del código moderno usa características de ES6+

| Versión      | Año           | Características                            |
|--------------|---------------|--------------------------------------------|
| ES5          | 2009          | Popularizó JavaScript                      |
| ES2015 (ES6) | 2015          | Clases, módulos, arrow functions, promises |
| ES2016+      | 2016-presente | Actualizaciones anuales con compatibilidad |



# JavaScript vs Java

## ⚠ ¡Importante!

Aunque la sintaxis recuerda a Java, son lenguajes **completamente diferentes**.

- **El nombre “JavaScript” fue puro marketing:** Se eligió para aprovechar la popularidad de Java
- **Inicialmente se llamó LiveScript:** Cambió de nombre antes de su lanzamiento oficial
- **Java estaba en auge** cuando se publicó JavaScript (1995)
- **Diferentes propósitos:** Java para aplicaciones robustas, JavaScript para web interactivo

# DOM y BOM

## DOM (Document Object Model)

- **Representa HTML como árbol de objetos:** Cada etiqueta HTML es un objeto manipulable
- **Modificación dinámica:** Cambia contenido sin recargar la página completa
- **Gestión de eventos:** Captura clicks, teclas, movimientos del ratón, etc.
- **Ejemplo:** `document.getElementById('boton')` accede a un elemento

## BOM (Browser Object Model)

- **Controla el navegador entero:** No solo el documento, sino ventanas, historial, URL
- **window.location:** Navega a otras URLs o recarga la página
- **window.history:** Retrocede/avanza en el historial del navegador
- **window.localStorage:** Guarda datos en el navegador
- **El BOM contiene al DOM:** `window.document` es el DOM



# Librerías JavaScript

## Para videojuegos:

- Phaser: Framework completo para juegos 2D - maneja física, colisiones, animaciones

## Herramientas de desarrollo:

- Webpack: Empaquetador de módulos - combina y optimiza tu código JavaScript y assets
- Babel: Transpilador - convierte código JavaScript moderno a versiones compatibles con navegadores antiguos

## Backend y comunicación:

- Express.js: Framework minimalista para crear servidores web y APIs en Node.js
- WS (ws.js): Biblioteca para WebSockets - permite comunicación en tiempo real bidireccional



# Configuración del Entorno

# Node.js

- **Ejecuta JavaScript fuera del navegador:** En tu ordenador, servidores, etc.
- **npm (Node Package Manager):** Gestor de paquetes para instalar librerías
- **Entorno de desarrollo moderno:** Necesario para usar herramientas como Webpack

Node.js permite ejecutar JavaScript fuera del navegador.

## Instalación:

1. Descargar desde [nodejs.org](https://nodejs.org)
2. Instalar versión LTS (Long Term Support - más estable)

## Verificar instalación:

```
1 # Muestra la versión de Node.js instalada
2 node --version
3
4 # Muestra la versión de npm (viene incluido con Node)
5 npm --version
```



# Inicializar un Proyecto

- **npm init -y**: Crea `package.json` con configuración por defecto (sin preguntas)
- **package.json**: Archivo que describe tu proyecto y sus dependencias
- **src/**: Carpeta con tu código fuente JavaScript
- **dist/**: Carpeta con archivos finales optimizados listos para producción
- **public/**: Recursos estáticos como HTML, imágenes, CSS que no se procesan

```
1 # Inicializar proyecto npm (crea package.json) dentro de una carpeta
2 npm init -y
```

## Estructura recomendada:

```
mi-juego-web/
├── package.json      # Configuración del proyecto
├── webpack.config.js # Configuración de Webpack
├── src/              # Tu código fuente
├── dist/             # Archivos compilados (generado)
├── public/           # HTML y recursos estáticos
│   └── index.html
```



# Instalación de Dependencias

- **Dependencias de producción:** Librerías que necesita tu juego para funcionar
- **Dependencias de desarrollo:** Herramientas solo para programar (no van al juego final)
- **npm install:** Descarga e instala paquetes desde el registro de npm
- **-save-dev:** Marca como dependencia de desarrollo

## Dependencias de producción:

```
1 # Phaser: motor de juegos 2D
2 # Lodash: utilidades para datos
3 # Axios: peticiones HTTP
4 npm install phaser lodash axios
```

## Dependencias de desarrollo:

```
1 # Webpack: empaqueta y optimiza código
2 npm install --save-dev webpack webpack-cli webpack-dev-server
3
4 # Plugins de Webpack para HTML y CSS
5 npm install --save-dev html-webpack-plugin css-loader style-loader
6
7 # Herramientas de calidad de código
8 npm install --save-dev eslint prettier
```



# Configuración de Webpack

- **Webpack**: Empaquetador que une todos tus archivos JS en uno solo optimizado
- **entry**: Punto de entrada - primer archivo que se ejecuta
- **output**: Dónde guardar el resultado final
- **plugins**: Extensiones que añaden funcionalidades (ej: generar HTML)
- **devServer**: Servidor de desarrollo con recarga automática

```
1  const path = require('path');
2  const HtmlWebpackPlugin = require('html-webpack-plugin');
3
4  module.exports = {
5    // **entry**: Punto de entrada de la aplicación
6    entry: './src/index.js',
7
8    // **output**: Dónde se guarda el bundle generado
9    output: {
10     path: path.resolve(__dirname, 'dist'),
11     filename: 'bundle.js',
12     clean: true // Limpia la carpeta 'dist' antes de cada build
13   },
14
15   // **plugins**: Añade funcionalidades extra (como generar el HTML automáticamente)
16   plugins: [
17     new HtmlWebpackPlugin({
18       template: './public/index.html', // Usa esta plantilla HTML
19       filename: 'index.html' // Nombre del archivo generado
20     })
21   ]
22 }
```



# Scripts de Desarrollo

- **Scripts npm:** Comandos personalizados para automatizar tareas comunes
- **npm run dev:** Inicia servidor de desarrollo con recarga automática
- **npm run build:** Crea versión optimizada para producción
- **npm run lint:** Revisa errores de código con ESLint
- **npm run format:** Formatea código con Prettier

## package.json:

```
1 {  
2   "scripts": {  
3     // Construye el paquete para producción usando Webpack  
4     "build": "webpack --mode production",  
5  
6     // Inicia el servidor de desarrollo con Webpack Dev Server y habilita hot-reload  
7     "dev": "webpack serve --mode development",  
8  
9     // Inicia el servidor en modo producción (requiere que server.js esté correctamente configurado)  
10    "start": "node server.js",  
11  
12    // Observa los cambios en los archivos y recompila automáticamente (sin servidor)  
13    "watch": "webpack --watch"  
14  }  
15 }
```



# El Lenguaje JavaScript

# Características del Lenguaje

- **Imperativo y Estructurado:** Escribe instrucciones paso a paso, como Java o C
- **Lenguaje de Script:** El navegador ejecuta el código directamente, sin compilar primero
- **Tipado Dinámico:** Las variables pueden cambiar de tipo ( `let x = 5; x = "texto";` )
- **Orientado a Objetos:** Usa prototipos en lugar de clases tradicionales (hasta ES6)
- **Funcional:** Las funciones son valores - pueden pasarse como argumentos



# Modo Estricto

- **Detecta errores que normalmente JavaScript ignora:** Convierte errores silenciosos en excepciones
- **Prohíbe sintaxis peligrosa:** Variables sin declarar, duplicar parámetros, etc.
- **Mejor rendimiento:** Permite optimizaciones del motor JavaScript
- **En módulos ES2015+ ya está activo:** No necesitas añadirlo manualmente
- También se puede utilizar `node --use_strict` en node.

```
1 // Activar modo estricto (poner al inicio del archivo o función)
2 "use strict";
3
4 // Ahora esto genera error (sin strict mode, crea variable global)
5 playerName = "Juan"; // Error: playerName is not defined
```

# Integración con HTML

- **Scripts en <head>** : Se ejecutan antes de cargar el contenido (puede bloquear renderizado)
- **Scripts al final de <body>** : Mejor práctica - el HTML ya está cargado
- **async**: Descarga el script en paralelo, ejecuta en cuanto esté listo (orden no garantizado)
- **defer**: Descarga en paralelo, pero ejecuta en orden después del HTML

```
1 <html>
2 <head>
3     <!-- Script en head - se ejecuta inmediatamente -->
4     <script src="js/config.js"></script>
5 </head>
6 <body>
7     <!-- Contenido HTML -->
8     <!-- Scripts al final - mejor rendimiento (HTML ya cargado) -->
9     <script src="js/game.js"></script>
10
11     <!-- async: descarga paralela, ejecuta en cuanto se descarga -->
12     <script src="js/game.js" async></script>
13
14     <!-- defer: descarga paralela, ejecuta después del HTML -->
15     <script src="js/game.js" defer></script>
16 </body>
17 </html>
```

# Mostrar Información

- **document.write()**: Escribe directamente en el HTML (evitar - obsoleto)
- **console.log()**: Muestra información en consola del navegador (F12)
- **console.error()**: Muestra errores en rojo - útil para debugging
- **console.warn()**: Muestra advertencias en amarillo

```
1 // Escribir en el documento HTML (no recomendado)
2 document.write('Texto');
3
4 // Consola del navegador (debugging) - Pulsa F12 para verla
5 console.log('Información de debug');
6 console.error('Error crítico');
7 console.warn('Advertencia');
8
9 // También puedes mostrar objetos
10 console.log('Jugador:', { nombre: 'Juan', vida: 100 });
```



# Comentarios

- Igual que en Java.

```
1 // Comentario de una línea - se ignora al ejecutar
2
3 /*
4  * Comentario
5  * multilínea
6  * útil para documentar funciones
7  */
8
9 // Los comentarios explican el "por qué", no el "qué"
10 // Malo: let x = 5; // Asigna 5 a x
11 // Bueno: let maxRetries = 5; // Límite de reintentos antes de fallar
```

# Variables

- **const**: Usa por defecto - evita modificaciones accidentales de valores que no deben cambiar
- **let**: Solo cuando necesites reasignar el valor (contadores, acumuladores)
- **var**: Obsoleto - tiene problemas de scope que causan bugs difíciles de encontrar
- **Block scope**: Variables solo existen dentro del bloque `{ }` donde se declaran

```
1 // const - valor inmutable (no se puede reasignar), block scope
2 const MAX_LIVES = 3;
3 const GAME_CONFIG = {
4     width: 800,
5     height: 600
6 };
7
8 // let - valor mutable (se puede reasignar), block scope
9 let currentLives = MAX_LIVES;
10 let playerPosition = { x: 0, y: 0 };
11
12 // var - evitar en código nuevo (function scope - problemas)
13 var oldStyle = "no recomendado";
14
15 // Ejemplos de uso
16 currentLives = 2; // OK - let permite reasignar
17 MAX_LIVES = 5; // ERROR - const no permite reasignar
```



# Ámbito de Variables

- **Block scope** ( `let` , `const` ): Variable solo existe dentro de las llaves `{}` donde se declaró
- **Function scope** ( `var` ): Variable existe en toda la función, ignorando bloques `{}`
- **Variables sin declarar**: Se vuelven globales (accesibles desde cualquier lugar) - PELIGROSO
- **Modo estricto**: Convierte variables sin declarar en error

```
1 function ejemploScope() {  
2     if (true) {  
3         let bloqueVariable = "solo aquí";  
4         var funcionVariable = "toda la función";  
5     }  
6  
7     // console.log(bloqueVariable); // ERROR - no existe fuera del if  
8     console.log(funcionVariable);    // OK - var tiene function scope  
9 }  
10  
11 // Sin modo estricto  
12 function peligro() {  
13     sinDeclarar = "ups"; // Crea variable global - BAD!  
14 }
```



# Tipos de Datos Primitivos

- **Number**: Un solo tipo para enteros y decimales (64 bits de precisión)
- **String**: Cadenas de texto - pueden usar `"""`, `' '` o `` `` (template literals)
- **Boolean**: Solo dos valores: `true` o `false`
- **null**: Ausencia intencional de valor - “esto está vacío a propósito”
- **undefined**: Variable declarada pero sin valor asignado - “aún no tiene valor”

```
1 // Number - enteros y decimales (punto flotante de 64 bits)
2 let score = 1000;
3 let health = 75.5;
4 let infinity = Infinity; // Valor especial
5
6 // String - cadenas de caracteres
7 let playerName = "Jugador1";
8
9 // Boolean - verdadero o falso
10 let isGameRunning = true;
11 let isPaused = false;
12
13 // Tipos especiales
14 let powerUp = null;           // Ausencia intencional - "no hay powerup"
15 let specialAbility;          // undefined - no inicializada aún
```



# Template Literals (ES2015+)

- **Backticks ( ` )**: En lugar de comillas normales
- **Interpolación \${}**: Inserta variables o expresiones dentro del string
- **Multilínea**: Puedes escribir strings en varias líneas sin concatenar
- **Expresiones**: Dentro de **\${}** puedes poner cualquier código JavaScript

```
1 let level = 5;
2 let experience = 1250;
3
4 // Interpolación de strings - inserta valores de variables
5 let status = `Nivel ${level} - EXP: ${experience}`;
6
7 // También puedes usar expresiones dentro de ${}
8 let progress = `Progreso: ${((experience / 2000) * 100)}%`;
9
10 // Strings multilínea - se respetan los saltos de línea
11 let gameInfo = `
12 Jugador: ${playerName}
13 Nivel: ${level}
14 Puntuación: ${score}
15 Estado: ${score > 1000 ? 'Pro' : 'Novato'}
16 `;
```



# Operadores Aritméticos

- **Operadores matemáticos básicos:** `+`, `-`, `*`, `/`, `%` (módulo/resto)
- **Comparación:** `>`, `<`, `>=`, `<=` comparan valores
- **Lógicos:** `&&` (y), `||` (o), `!` (no) para combinar condiciones
- **Precedencia:** Multiplicación y división antes que suma y resta

```
1 // Operadores aritméticos - similares a Java
2 let damage = baseDamage + bonus;
3 let remaining = total - used;
4 let area = width * height;
5 let average = sum / count;
6 let remainder = value % modulo; // Resto de división
7
8 // Lógicos - combinan condiciones -> Devuelven true or false
9 let canAct = isAlive && !isStunned; // Y lógico, NO lógico
10 let shouldRespawn = isDead || health <= 0; // 0 lógico
```



# Operadores de Comparación

- **=== (estricto)**: Compara valor Y tipo - **siempre recomendado**
- **== (débil)**: Convierte tipos antes de comparar - **evitar** (causa bugs)
- **!== (desigualdad estricta)**: Diferente valor O tipo
- **Ejemplo**: `5 == "5"` es `true`, pero `5 === "5"` es `false`

```
1 // Igualdad estricta (recomendado) - compara valor Y tipo
2 if (playerID === targetID) {
3     // Solo true si ambos tienen el mismo valor Y tipo
4 }
5
6 // Desigualdad estricta - diferente valor O tipo
7 if (level !== previousLevel) {
8     // true si son diferentes
9 }
10
11 // Igualdad débil (EVITAR) - hace conversión de tipos
12 if (score == "100") {
13     // true - convierte "100" a número 100. Puede causar bugs difíciles de encontrar
14 }
15
16 // Comparación de tipos diferentes
17 console.log(5 === "5"); // false - número vs string
18 console.log(5 == "5"); // true - convierte a mismo tipo
```



# Operadores Modernos (ES2020+)

- **Nullish coalescing ( ?? )**: Valor por defecto solo si `null` o `undefined`
- **Optional chaining ( ?. )**: Accede a propiedades sin error si no existen
- **Logical assignment**: Asigna solo si cumple condición
- **Diferencia con ||**: `||` también considera `0`, `""`, `false` como “falsy”

```
1 // Nullish coalescing - valor por defecto solo para null/undefined
2 let playerName = savedName ?? "Jugador Anónimo";
3 // Si savedName es 0 o "" NO usa el default (solo null/undefined)
4
5 // Optional chaining - evita errores si propiedades no existen
6 let weapon = player.inventory?.equipment?.weapon;
7 // Si inventory o equipment es null/undefined, retorna undefined sin error
8
9 // Logical assignment
10 playerName ||= "Jugador por defecto"; // Asigna solo si falsy
11 playerName ??= "Valor por defecto";   // Asigna solo si null/undefined
12
13 // Diferencia entre || y ??
14 let count = 0;
15 let result1 = count || 10; // 10 (0 es falsy)
16 let result2 = count ?? 10; // 0 (0 no es null/undefined)
```



# Valores Falsy

- JavaScript evalúa algunos valores como **false** en condiciones
- 6 valores falsy: `false` , `0` , `""` , `null` , `undefined` , `NaN`
- Todos los demás son **truthy**: Incluso `"0"` , `[]` , `{}`
- **Útil para**: Validaciones y valores por defecto

JavaScript considera **falso** (falsy):

- `false` - booleano falso
- `null` - sin valor
- `undefined` - no definido (verificación de existencia)
- `""` - string vacío
- `0` - cero numérico
- `NaN` - “Not a Number”



# Utiliza siempre ===

|           | true | false | 1 | 0 | -1 | "true" | "false" | "1" | "0" | "-1" | "" | null | undefined | Infinity | -Infinity | [] | {} | [[] ] | [0] | [1] | NaN |
|-----------|------|-------|---|---|----|--------|---------|-----|-----|------|----|------|-----------|----------|-----------|----|----|-------|-----|-----|-----|
| true      |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| false     |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| 1         |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| 0         |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| -1        |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| "true"    |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| "false"   |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| "1"       |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| "0"       |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| "-1"      |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| ""        |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| null      |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| undefined |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| Infinity  |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| -Infinity |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| []        |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| {}        |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| [[]]      |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| [0]       |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| [1]       |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |
| NaN       |      |       |   |   |    |        |         |     |     |      |    |      |           |          |           |    |    |       |     |     |     |

# Arrays

# Creación de Arrays

- **Arrays dinámicos:** Pueden crecer o reducirse automáticamente
- **Tipos mezclados:** Puedes guardar diferentes tipos en el mismo array
- **Índice desde 0:** El primer elemento está en posición 0
- **length:** Propiedad que indica cuántos elementos tiene

```
1 // Creación de arrays - literal (forma más común)
2 let empty = [];
3 let numbers = [1, 2, 3, 4, 5];
4 let mixed = ["texto", 42, true, null]; // Tipos mezclados OK
5
6 // Constructor Array (menos común)
7 let inventory = new Array(10); // Crea array con 10 espacios vacíos
8
9 // Acceso y modificación por índice
10 console.log(numbers[0]); // 1 - primer elemento
11 numbers[2] = 999; // Modifica tercer elemento
12
13 // Arrays dinámicos - se expanden automáticamente
14 numbers[10] = 100; // Salta del índice 4 al 10
15 console.log(numbers.length); // 11 - ahora tiene 11 elementos
16 // Los índices 5-9 quedan vacíos (undefined)
```



# Métodos de Array Modernos

- **filter()**: Crea nuevo array con elementos que cumplen condición
- **map()**: Crea nuevo array transformando cada elemento
- **No modifican el original**: Retornan nuevo array (inmutabilidad)
- **Callback**: Función que se ejecuta por cada elemento

```
1 // Array de ejemplo
2 let enemies = [
3   { id: 1, health: 100, type: "orc" },
4   { id: 2, health: 50, type: "goblin" },
5   { id: 3, health: 0, type: "orc" }
6 ];
7
8 // filter() - selecciona elementos que cumplen condición
9 // Retorna nuevo array con enemigos vivos
10 let aliveEnemies = enemies.filter(enemy => enemy.health > 0);
11 // [{ id: 1, ... }, { id: 2, ... }]
12
13 // Solo los orcos
14 let orcs = enemies.filter(enemy => enemy.type === "orc");
15
16 // map() - transforma cada elemento
17 // Extrae solo la salud de cada enemigo
18 let healthValues = enemies.map(enemy => enemy.health);
19 // [100, 50, 0]
```



# Más Métodos de Arrays

- **find()**: Retorna el **primer** elemento que cumple condición
- **findIndex()**: Retorna el **índice** del primer elemento que cumple condición
- **every()**: Retorna **true** si **todos** los elementos cumplen condición
- **some()**: Retorna **true** si **algún** elemento cumple condición
- **reduce()**: Reduce array a un único valor (suma, acumulación, etc.)

```
1 // find() - retorna PRIMER elemento que cumple condición
2 let firstOrc = enemies.find(enemy => enemy.type === "orc"); // { id: 1, health: 100, type: "orc" }
3
4 // findIndex() - retorna índice del primer match
5 let orcIndex = enemies.findIndex(enemy => enemy.type === "orc"); // 0
6
7 // every() - ¿TODOS cumplen la condición?
8 let allDead = enemies.every(enemy => enemy.health === 0); // false -> no todos muertos
9
10 // some() - ¿ALGUNO cumple la condición?
11 let someDead = enemies.some(enemy => enemy.health === 0); // true -> al menos un muerto
12
13 // reduce() - reduce array a un solo valor
14 // Suma total de la salud de todos los enemigos
15 let totalHealth = enemies.reduce((sum, enemy) =>
16     sum + enemy.health, 0); // 0 es valor inicial
17 // 150 (100 + 50 + 0)
```



# Modificar Arrays

- **push()**: Añade elemento al final - modifica el array original
- **pop()**: Elimina y retorna último elemento
- **shift()**: Elimina y retorna primer elemento
- **unshift()**: Añade elemento al principio
- **splice()**: Elimina/inserta elementos en cualquier posición

```
1 // push() - añadir al final (modifica original)
2 enemies.push({ id: 4, health: 75, type: "troll" });
3
4 // shift() - quitar del principio
5 let firstEnemy = enemies.shift();
6
7 // pop() - quitar del final
8 let lastEnemy = enemies.pop();
9
10 // splice(inicio, cantidad) - eliminar elementos
11 enemies.splice(1, 2); // Eliminar 2 elementos desde índice 1
12
13 // splice(inicio, 0, elemento) - insertar sin eliminar
14 enemies.splice(1, 0, newEnemy); // Insertar en índice 1
15
16 // unshift() - añadir al principio
17 enemies.unshift({ id: 0, health: 50, type: "scout" });
```



# Destructuring de Arrays

- **Destructuring:** Extraer valores de arrays a variables individuales
- **Rest operator ( ... ):** Captura “el resto” de elementos
- **Sintaxis limpia:** Evita acceder por índice repetidamente

```
1 // Extraer valores de array a variables
2 let coordinates = [100, 200];
3 let [x, y] = coordinates; // x = 100, y = 200
4
5 // Ignorar elementos
6 let [first, , third] = [1, 2, 3]; // first = 1, third = 3
7
8 // Rest operator - captura el resto de elementos
9 let [firstItem, secondItem, ...rest] = inventory;
10 // firstItem = primer elemento
11 // secondItem = segundo elemento
12 // rest = array con todos los demás
13
14 // En parámetros de función
15 function showCoords([x, y]) {
16     console.log(`X: ${x}, Y: ${y}`);
17 }
```



# Control de Flujo

# Sentencias Básicas

- **if-else**: Ejecuta código según condición verdadera o falsa
- **else if**: Permite múltiples condiciones en cadena
- **Condiciones**: Expresiones que evalúan a `true` o `false`
- **Bloques {}** : Agrupan varias instrucciones

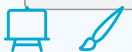
```
1 // if - else - estructura condicional básica
2 if (health > 50) { // Siempre recomendado con llaves
3     // Se ejecuta si la condición es verdadera
4     statusColor = "green";
5 } else if (health > 20) {
6     // Se ejecuta si la primera es falsa y esta es verdadera
7     statusColor = "yellow";
8 } else {
9     // Se ejecuta si todas las anteriores son falsas
10    statusColor = "red";
11 }
12
13 // Sin llaves para una sola instrucción (no recomendado)
14 if (isDead) gameOver();
```



# Switch

- **switch**: Compara una expresión contra múltiples valores
- **case**: Cada posible valor a comparar
- **break**: Sale del switch (sin él, continúa al siguiente case)
- **default**: Se ejecuta si ningún case coincide

```
1 // switch - útil cuando comparas misma variable con muchos valores
2 switch (gameState) {
3     case "menu":
4         // Se ejecuta si gameState === "menu"
5         showMenu();
6         break; // Sale del switch.
7
8     case "rollback": // Como no hay break se ejecuta el siguiente
9         rollback();
10
11     case "playing":
12         updateGame();
13         break;
14
15     default:
16         // Se ejecuta si ningún case coincide
17         handleUnknownState();
18 }
```



# Loops

- **for tradicional**: Cuando sabes cuántas iteraciones necesitas
- **for...of**: Itera sobre valores de un array (ES2015+) - más limpio
- **for...in**: Itera sobre claves/propiedades de un objeto
- **break**: Sale del loop inmediatamente
- **continue**: Salta a la siguiente iteración

```
1 // for tradicional - control total sobre índice
2 for (let i = 0; i < enemies.length; i++) {
3     updateEnemy(enemies[i]);
4     // i = 0, 1, 2, ... hasta length-1
5 }
6
7 // for...of - itera valores (ES2015+) - MÁS LIMPIO
8 for (let enemy of enemies) {
9     updateEnemy(enemy); // enemy es el valor directamente
10 }
11
12 // for...in - itera propiedades/claves (para objetos)
13 for (let key in gameConfig) {
14     console.log(key, gameConfig[key]);
15     // key = "width", "height", etc.
16 }
```



# While

- **while**: Repite mientras la condición sea verdadera
- **Cuidado con loops infinitos**: Asegúrate que la condición eventualmente sea falsa
- **do...while**: Ejecuta al menos una vez, luego verifica condición

```
1 // while - repite mientras la condición sea true
2 while (isGameRunning && playerLives > 0) {
3     processGameFrame();
4     // IMPORTANTE: algo dentro debe cambiar la condición
5     // o será un loop infinito
6 }
7
8 // do...while - ejecuta AL MENOS una vez
9 let input;
10 do {
11     input = prompt("Ingresa comando:");
12 } while (input !== "quit");
```

# Funciones

# Declaración de Funciones

- **function**: Palabra clave para declarar funciones
- **Parámetros**: Valores que recibe la función (entre paréntesis)
- **return**: Devuelve un valor y termina la función
- **Expresión de función**: Asignar función a una variable
- **Hoisting**: Las declaraciones se mueven al inicio (se pueden llamar antes de declarar)

```
1 // Declaración tradicional - se puede llamar antes de declarar
2 function calculateDamage(baseDamage, criticalHit) {
3     // Verifica si hay golpe crítico
4     if (criticalHit) {
5         return baseDamage * 2; // Retorna el doble
6     }
7     return baseDamage; // Retorna daño normal
8 }
9
10 // Uso
11 let damage = calculateDamage(50, true); // 100
```

```
1 // Expresión de función - asignada a variable
2 let heal = function(amount) {
3     player.health += amount; // Suma salud
4     // Limita la salud al máximo
5     if (player.health > player.maxHealth) {
6         player.health = player.maxHealth;
7     }
8 };
9
10 // Uso
11 heal(20);
```



# Arrow Functions (ES2015+)

- **Sintaxis más concisa** que funciones tradicionales
- **No tienen su propio `this`** : Heredan `this` del contexto (útil en callbacks)
- **Retorno implícito**: Si es una sola expresión, retorna automáticamente (sin `return` )
- **Ideal para**: Callbacks, funciones cortas, map/filter/reduce

```
1 // Arrow function completa con llaves
2 let movePlayer = (deltaX, deltaY) => {
3     player.x += deltaX; // Mueve en X
4     player.y += deltaY; // Mueve en Y
5 };
6
7 // Arrow function con una expresión - retorno implícito
8 let isAlive = (entity) => entity.health > 0;
9 // Equivalente a: function(entity) { return entity.health > 0; }
10
11 // Sin parámetros - paréntesis vacíos
12 let generateRandomID = () => Math.random().toString(36).slice(2);
13
14 // Un parámetro - paréntesis opcionales
15 let double = x => x * 2;
```



# Parámetros de Función

- **Parámetros por defecto:** Valor asignado si no se pasa argumento
- **Rest parameters ( ... ):** Captura argumentos restantes en un array
- **Destructuring:** Extrae propiedades de objetos directamente en parámetros
- **Orden:** Parámetros normales, luego con default, luego rest

```
1 // Parámetros por defecto (ES2015+)
2 function createEnemy(health = 100, damage = 10) {
3     return { health, damage };
4 }
5 // Si no pasas argumentos, usa los valores por defecto
6 createEnemy(); // { health: 100, damage: 10 }
7 createEnemy(50); // { health: 50, damage: 10 }
8
9 // Rest parameters - captura argumentos restantes en array
10 function logMessage(level, ...messages) {
11     console.log(`[${level}]`, ...messages);
12 }
13 logMessage("ERROR", "Falló", "al cargar", "recurso");
14 // messages = ["Falló", "al cargar", "recurso"]
```



# Closures

- **Closure**: Función que “recuerda” variables de su contexto exterior
- **Encapsulación**: Variables privadas que solo la función puede modificar
- **Estado persistente**: Mantiene estado entre llamadas sin variables globales
- **Útil para**: Contadores, factory functions, datos privados

```
1 // Closure - función que retorna objeto con métodos
2 function createCounter(initialValue = 0) {
3   // Variable privada - solo accesible dentro
4   let count = initialValue;
5
6   // Retorna objeto con métodos que acceden
7   // a 'count'
8   return {
9     // Incrementa y retorna nuevo valor
10    increment: () => ++count,
11
12    // Decrementa y retorna nuevo valor
13    decrement: () => --count,
14
15    // Retorna valor actual
16    getValue: () => count
17  };
18 }
```

```
1 // Cada contador tiene su propio estado
2 let scoreCounter = createCounter(0);
3 scoreCounter.increment(); // 1
4 scoreCounter.increment(); // 2
5 console.log(scoreCounter.getValue()); // 2
6
7 // 'count' NO es accesible desde fuera
8 // console.log(count);
9 // ERROR - no existe aquí
```



# Manejo de Excepciones

# Try-Catch-Finally

- **try**: Bloque donde puede ocurrir un error
- **catch**: Captura el error si ocurre y maneja la situación
- **finally**: Se ejecuta SIEMPRE (haya error o no) - útil para limpieza
- **error.message**: Descripción del error

```
1 // Manejo de errores al cargar partida guardada
2 try {
3     // Intenta parsear JSON - puede fallar si está corrupto
4     let gameData = JSON.parse(savedGameString);
5     loadGame(gameData); // Carga el juego
6
7 } catch (error) {
8     // Se ejecuta SOLO si hay error en try
9     console.error('Error loading game:', error.message);
10    showErrorDialog('No se pudo cargar la partida');
11
12 } finally { // Opcional
13     // Se ejecuta SIEMPRE (con o sin error)
14     hideLoadingSpinner();
15 }
```



# Lanzar Excepciones

- **throw**: Lanza un error manualmente
- **new Error()**: Crea objeto de error con mensaje
- **Validaciones**: Úsalo para validar inputs antes de procesar
- **El error sube**: Si no hay catch, el error se propaga hacia arriba

```
1 // Función que valida input y lanza errores si es inválido
2 function validatePlayerInput(input) {
3     // Verifica que no esté vacío
4     if (!input || input.trim() === '') {
5         throw new Error('El nombre no puede estar vacío');
6     }
7
8     // Verifica longitud máxima
9     if (input.length > 20) {
10        throw new Error('El nombre es demasiado largo');
11    }
12
13    // Si pasa las validaciones, retorna input limpio
14    return input.trim();
15 }
```



# Almacenamiento de Datos

# Local Storage

- **localStorage**: Almacena datos en el navegador de forma permanente
- **Capacidad**: ~5-10MB (depende del navegador)
- **Solo strings**: Debes convertir objetos a JSON con `JSON.stringify()`
- **Persiste**: Datos permanecen incluso cerrando navegador/reiniciando PC
- **Por dominio**: Cada sitio web tiene su propio localStorage

```
1 // Guardar datos simples (solo strings)
2 localStorage.setItem('playerName', 'Jugador1');
3 localStorage.setItem('highScore', '15000');
4
5 // Guardar objetos - primero convertir a JSON string
6 const gameConfig = { volume: 0.8, difficulty: 'normal' };
7 localStorage.setItem('gameConfig', JSON.stringify(gameConfig));
8
9 // Leer datos simples
10 const playerName = localStorage.getItem('playerName');
11 // "Jugador1"
12
13 // Eliminar un dato específico
14 localStorage.removeItem('playerName');
15
16 // Eliminar TODOS los datos (usar con cuidado)
17 localStorage.clear();
```



# Session Storage

- **sessionStorage**: API idéntica a localStorage
- **Duración**: Solo durante la sesión actual (cerrar pestaña = se borra)
- **Por pestaña**: Cada pestaña tiene su propio sessionStorage
- **Uso típico**: Datos temporales como estado actual del juego
- **No se comparte**: Entre pestañas ni ventanas

```
1 // API idéntica a localStorage
2 sessionStorage.setItem('tempData', 'valor temporal');
3 const tempData = sessionStorage.getItem('tempData');
4 sessionStorage.removeItem('tempData');
5 sessionStorage.clear();
6
7 // Ideal para estado temporal del juego actual
8 sessionStorage.setItem('currentGameState',
9     JSON.stringify(gameState));
```



# Cookies

- **Cookies:** Método antiguo de almacenamiento
- **Capacidad limitada:** Solo 4KB
- **Se envían al servidor:** En cada petición HTTP (aumenta tráfico)
- **API manual:** Más complicado que localStorage
- **Expiración configurable:** Puedes definir cuándo expiran

```
1 // Escribir Cookie con expiración (30 días desde ahora)
2 // Sin fecha es la sesión
3 const fecha = new Date();
4 fecha.setTime(fecha.getTime() + (30 * 24 * 60 * 60 * 1000));
5 document.cookie = `highScore=15000; expires=${fecha.toUTCString()}; path=/`;
6
7 // Leer cookie (complicado - no hay API directa)
8 function getCookie(nombre) {
9     const value = `; ${document.cookie}`;
10    const parts = value.split(`; ${nombre}=`);
11    if (parts.length === 2) return parts.pop().split(';').shift();
12    return null; // No encontrada
13 }
14
15 // Uso
16 let highScore = getCookie('highScore'); // "15000"
17
18 // Eliminar cookie (establecer fecha pasada)
19 document.cookie = "playerName=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/";
```



# Comparación de Métodos

- **localStorage**: Mejor opción para datos persistentes (configuración, progreso)
- **sessionStorage**: Perfecto para datos temporales de la sesión actual
- **Cookies**: Solo si necesitas comunicar con el servidor

| Característica    | localStorage               | sessionStorage     | Cookies                |
|-------------------|----------------------------|--------------------|------------------------|
| Capacidad         | ~5-10MB                    | ~5-10MB            | 4KB                    |
| Persistencia      | Hasta eliminar manualmente | Solo sesión actual | Configurable (expires) |
| Envío al servidor | No                         | No                 | Sí (automático)        |
| API               | Síncrona y simple          | Síncrona y simple  | Manual y compleja      |
| Compartido        | Entre pestañas             | NO (por pestaña)   | Entre pestañas         |

